

LABORATORY RECORD

Course: Full Stack Web Development

Course Code: 23CM4121

Experiment No.: 4

Student Name: K. SUHAS VARMA

Roll No.: A24126552091

Date: 29-08-2026

1. TITLE

JavaScript Basics – Arrays & Functions

2. AIM

To understand and implement the basic concepts of JavaScript arrays, functions, loops, conditional statements and return values by creating a simple program that processes an array of numbers and observing its output.

3. OBJECTIVE

The objectives of this experiment are:

- To understand the concept of arrays in JavaScript.
- To create and initialize an array with numeric values.
- To access array elements using index numbers and find the length of an array.
- To iterate through an array using the for loop.
- To use the push() method to add elements to an array.
- To create reusable functions that accept an array as an argument.
- To return values (a number and arrays) from functions using the return statement.
- To use arithmetic operators (+, *, %) and comparison operators (==, >=).
- To use the if...else conditional statement to make decisions.
- To display output using console.log().

4. THEORY

JavaScript is a high-level programming language commonly used to add dynamic behavior and interactivity to web applications. It supports concepts such as variables, arrays, functions, loops, conditional statements and operators.

Arrays

An array is a collection of multiple values stored in a single variable. Array elements are accessed using an index, which starts from 0. For example: `const numbers = [10, 20, 35, 40, 50]`; Here, 10 is at index 0, 20 is at index 1, and 50 is at index 4. Useful array features used in this experiment are:

- `length` – returns the number of elements in an array.
- `push()` – adds an element to the end of an array.
- `arr[i]` – accesses the element at index i.

Loops

A for loop repeats a block of code a fixed number of times. It has an initialization (`let i = 0`), a condition (`i < arr.length`) and an update (`i++`). In this experiment the for loop is used to visit every element of the array.

Functions

A function is a reusable block of code designed to perform a particular task. Functions can accept parameters, can be called multiple times with different values, and can send a result back using the `return` statement. In this experiment four functions are used:

- `getTotal(arr)` – adds all elements and returns the sum.
- `doubleNumbers(arr)` – multiplies every element by 2 and returns a new array.
- `evennumbers(arr)` – checks each element with the modulus operator (%) and returns an array of true/false values.
- `checkscores(arr)` – prints "pass" if a score is 35 or more, otherwise prints "fail".

Operators and Conditions

The modulus operator % gives the remainder of a division; a number is even when `n % 2 == 0` is true. The `if...else` statement executes one block of code when the condition is true and another block when it is false.

Points to Note

- In `checkscores()`, the loop variable `i` is declared without `let`, so it becomes an implicit global variable (this works in normal, non-strict mode, but using `let i` is the recommended practice).
- The function `checkscores` returns its own name (the function itself) instead of a value, so `console.log(checkscores(numbers))` prints `[Function: checkscores]` after the pass/fail lines.

The basic concepts demonstrated in this experiment are:

Arrays → Loops → Functions → Return Values → Operators → Conditions → Console Output

5. ALGORITHM / PROCEDURE

1. Create a project folder and a JavaScript file named `script.js`.
2. Create an array named `numbers` containing the values 10, 20, 35, 40 and 50.
3. Define the function `getTotal(arr)`: initialize sum to 0, add each element using a for loop, and return sum.
4. Define the function `doubleNumbers(arr)`: create an empty array, push each element multiplied by 2, and return the new array.
5. Define the function `evennumbers(arr)`: create an empty array, push the result of `arr[i] % 2 == 0` for each element, and return the array.
6. Define the function `checkscores(arr)`: for each element, print "pass" if it is greater than or equal to 35, otherwise print "fail".
7. Display the original array using `console.log()`.
8. Call `getTotal(numbers)` and display the total sum.
9. Call `doubleNumbers(numbers)` and display the doubled array.
10. Call `evennumbers(numbers)` and display the array of true/false values.
11. Call `checkscores(numbers)` and display its pass/fail messages and returned value.
12. Execute the program using Node.js with the command: `node script.js`.
13. Observe and record the console output.

6. PROGRAM

script.js

```
const numbers = [10, 20, 35, 40, 50];
function getTotal(arr) {
  let sum = 0;
  for (let i = 0; i < arr.length; i++) {
    sum += arr[i];
  }
  return sum;
}
function doubleNumbers(arr) {
  let doubled = [];
  for (let i = 0; i < arr.length; i++) {
    doubled.push(arr[i] * 2);
  }
  return doubled;
}
function evennumbers(arr){
  let evennum=[];
  for(let i=0;i<arr.length;i++){
    evennum.push(arr[i]%2==0);
  }
  return evennum;
}
function checkscores(arr){
  for(i=0;i<arr.length;i++){
    if(arr[i]>=35){
      console.log("pass");
    }
    else{
      console.log('fail');
    }
  }
  return checkscores;
}
console.log("Original numbers:", numbers);
console.log("Total sum:", getTotal(numbers));
console.log("Doubled numbers:", doubleNumbers(numbers));
console.log("Evennumbers:", evennumbers(numbers));
console.log(checkscores(numbers));
```

7. CODE EXPLANATION

Code / Concept	Explanation
<code>const numbers = [...]</code>	Creates an array containing the initial numeric values and stores it in a constant.
<code>function getTotal(arr)</code>	Defines a function that receives an array and returns the sum of its elements.
<code>let sum = 0</code>	Declares a variable to hold the running total.
<code>for (let i = 0; ...)</code>	Loop that visits every index of the array from 0 to length - 1.
<code>arr.length</code>	Returns the total number of elements in the array.
<code>arr[i]</code>	Accesses the element stored at index i.
<code>sum += arr[i]</code>	Adds the current element to the running total.
<code>return sum</code>	Sends the calculated value back to the caller and exits the function.
<code>function doubleNumbers(arr)</code>	Defines a function that returns a new array with every element doubled.
<code>let doubled = []</code>	Creates an empty array to store the results.
<code>doubled.push(arr[i] * 2)</code>	Multiplies the element by 2 and adds it to the end of the new array.
<code>function evennumbers(arr)</code>	Defines a function that checks whether each element is even.
<code>arr[i] % 2 == 0</code>	Modulus operator gives the remainder; the result is true when the number is even.
<code>function checkscores(arr)</code>	Defines a function that prints pass or fail for every score.
<code>if (arr[i] >= 35) ... else</code>	Prints "pass" when the score is 35 or more; otherwise executes the else block and prints "fail".
<code>for (i = 0; ...)</code>	Loop variable declared without let, creating an implicit global variable.
<code>return checkscores</code>	Returns the function itself, so printing the call shows [Function: checkscores].
<code>console.log()</code>	Displays output in the console.
<code>getTotal(numbers)</code>	Function call that passes the numbers array as an argument.

8. TEST CASES AND EXECUTION DATA

The JavaScript program was executed using Node.js and the console output was verified for every function. In the outputs below, the lines relevant to each test case are highlighted.

Test Case	TC-01
Test / Input	Execute script.js using Node.js (node script.js)
Expected Result	The program should run without errors and display the original array, total sum, doubled numbers, even-number results and pass/fail messages.
Actual Result	The program ran without any error and displayed all the expected outputs in the console.
Status	Pass

TC-01 Output:

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

$ node script.js
Original numbers: [ 10, 20, 35, 40, 50 ]
Total sum: 155
Doubled numbers: [ 20, 40, 70, 80, 100 ]
Evennumbers: [ true, true, false, true, true ]
fail
fail
pass
pass
pass
[Function: checkscores]
$ █
```

Test Case	TC-02
Test / Input	Create the array [10, 20, 35, 40, 50] and display it
Expected Result	The original array should be displayed as [10, 20, 35, 40, 50].
Actual Result	The console displayed: Original numbers: [10, 20, 35, 40, 50].
Status	Pass

TC-02 Output:

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

$ node script.js
Original numbers: [ 10, 20, 35, 40, 50 ]
Total sum: 155
Doubled numbers: [ 20, 40, 70, 80, 100 ]
Evennumbers: [ true, true, false, true, true ]
fail
fail
pass
pass
pass
[Function: checkscores]
$ █
```

Test Case	TC-03
Test / Input	Call getTotal(numbers)
Expected Result	The function should return the sum of all elements: $10 + 20 + 35 + 40 + 50 = 155$.
Actual Result	The console displayed: Total sum: 155.
Status	Pass

TC-03 Output:

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

$ node script.js
Original numbers: [ 10, 20, 35, 40, 50 ]
Total sum: 155
Doubled numbers: [ 20, 40, 70, 80, 100 ]
Evennumbers: [ true, true, false, true, true ]
fail
fail
pass
pass
pass
[Function: checkscores]
$ 

```

Test Case	TC-04
Test / Input	Call doubleNumbers(numbers)
Expected Result	A new array with every element multiplied by 2 should be returned: [20, 40, 70, 80, 100].
Actual Result	The console displayed: Doubled numbers: [20, 40, 70, 80, 100].
Status	Pass

TC-04 Output:

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

$ node script.js
Original numbers: [ 10, 20, 35, 40, 50 ]
Total sum: 155
Doubled numbers: [ 20, 40, 70, 80, 100 ]
Evennumbers: [ true, true, false, true, true ]
fail
fail
pass
pass
pass
[Function: checkscores]
$ 

```

Test Case	TC-05
Test / Input	Call evennumbers(numbers)
Expected Result	An array of true/false values should be returned: true for even numbers and false for odd numbers.
Actual Result	The console displayed: Evennumbers: [true, true, false, true, true]. Only 35 is odd, so it gives false.
Status	Pass

TC-05 Output:

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

$ node script.js
Original numbers: [ 10, 20, 35, 40, 50 ]
Total sum: 155
Doubled numbers: [ 20, 40, 70, 80, 100 ]
Evennumbers: [ true, true, false, true, true ]
fail
fail
pass
pass
pass
[Function: checkscores]
$ 

```

Test Case	TC-06
Test / Input	Call checkscores(numbers)
Expected Result	Scores below 35 should print "fail" and scores of 35 or more should print "pass" (10 → fail, 20 → fail, 35 → pass, 40 → pass, 50 → pass).
Actual Result	The console displayed fail, fail, pass, pass, pass in the same order as the array elements.
Status	Pass

TC-06 Output:

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

$ node script.js
Original numbers: [ 10, 20, 35, 40, 50 ]
Total sum: 155
Doubled numbers: [ 20, 40, 70, 80, 100 ]
Evennumbers: [ true, true, false, true, true ]
fail
fail
pass
pass
pass
[Function: checkscores]
$ 

```

Test Case	TC-07
Test / Input	Print the value returned by checkscores(numbers) using console.log()
Expected Result	Since the function returns checkscores (the function itself), the console should display [Function: checkscores].
Actual Result	The console displayed: [Function: checkscores] after the pass/fail messages.
Status	Pass

TC-07 Output:

```

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

$ node script.js
Original numbers: [ 10, 20, 35, 40, 50 ]
Total sum: 155
Doubled numbers: [ 20, 40, 70, 80, 100 ]
Evennumbers: [ true, true, false, true, true ]
fail
fail
pass
pass
pass
pass
[Function: checkscores]
$

```

9. OUTPUT

The JavaScript program was executed successfully. The corresponding console output is shown below.

Output: Array Functions – script.js

```

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

$ node script.js
Original numbers: [ 10, 20, 35, 40, 50 ]
Total sum: 155
Doubled numbers: [ 20, 40, 70, 80, 100 ]
Evennumbers: [ true, true, false, true, true ]
fail
fail
pass
pass
pass
pass
[Function: checkscores]
$

```

Figure 1: Output of Array Functions using JavaScript

The output demonstrates displaying the original array, calculating the total sum, doubling every element, checking even numbers, and printing pass/fail results for each score.

10. RESULT

Thus, the basic concepts of **JavaScript arrays and functions** were successfully implemented and executed. Array operations such as indexing, length, push() and iteration using the for loop were demonstrated. Functions were used to calculate the total, double the numbers, identify even numbers and check pass/fail scores using the modulus operator and if...else statement. The program was executed successfully and the expected outputs were obtained.